

RELATÓRIO DO LABORATÓRIO PRÁTICO

Trilha DevOps — CI/CD: GitHub Actions + Magalu Cloud Container Registry

Aluno: sergio-l-dev-mglcloud

Data: Julho de 2026

1. OBJETIVO DO LABORATÓRIO

Neste laboratório, foi construída uma esteira de Integração Contínua (CI) utilizando GitHub Actions integrada ao Magalu Cloud Container Registry.

Durante a atividade, foi desenvolvida uma pequena aplicação em Python, criada uma imagem Docker e configurado um workflow que automatiza todo o processo de build e publicação da imagem sempre que um git push é realizado no repositório.

Ao final do laboratório, foram praticados os seguintes conceitos:

- Configuração de um Container Registry na Magalu Cloud;
- Utilização de GitHub Secrets para armazenamento seguro de credenciais;
- Construção de imagens Docker;
- Criação de workflows com GitHub Actions;
- Publicação automática de imagens Docker no Container Registry da Magalu Cloud;
- Análise da execução da pipeline e validação das imagens publicadas.

2. AMBIENTE UTILIZADO

- Sistema Operacional: Windows (máquina local) + Ubuntu (VM docker-microservicos)
- Controle de Versão: Git + GitHub

- Containerização: Docker
- Registry: Magalu Cloud Container Registry (região br-se1)
- CI/CD: GitHub Actions
- Linguagem: Python 3.12 + Flask

3. MISSÕES EXECUTADAS

Missão 1 — Criando o Container Registry

- Acesso ao console da Magalu Cloud (console.magalu.cloud).
- Criação do registry com o nome "meu-registry-01" na região "br-se1".
- Geração das credenciais via CLI com o comando: `mgc container-registry credentials get`.
- Anotação do endpoint: `container-registry.br-se1.magalu.cloud`.

Missão 2 — Criando o Repositório GitHub

- Criação do repositório "meu-pipeline" no GitHub.
- Clone do repositório para a máquina local.
- Validação do remoto com o comando: `git remote -v`.

Missão 3 — Desenvolvendo a Aplicação

- Criação da aplicação Flask (app.py) com dois endpoints:
 - GET / -> retorna `{"application": "Meu Pipeline", "version": "1.0.0", "provider": "Magalu Cloud"}`
 - GET /health -> retorna `{"status": "UP"}`
- Criação do requirements.txt (Flask 3.0.3).
- Criação do Dockerfile baseado em `python:3.12-slim`.
- Criação do .dockerignore para excluir arquivos desnecessários do build.

Missão 4 — Construindo a Imagem Localmente

- Build da imagem com o comando: `docker build -t meu-pipeline:1.0.0` .
- Execução do container com o comando: `docker run -d -p 5000:5000 --name app-teste meu-pipeline:1.0.0`
- Validação dos endpoints via navegador (`http://localhost:5000` e `http://localhost:5000/health`) para garantir que a aplicação estava íntegra antes da automação.

Missão 5 — Configurando GitHub Secrets

- Criação dos secrets no repositório GitHub em Settings > Secrets and variables > Actions:
 - MAGALU_REGISTRY_USERNAME
 - MAGALU_REGISTRY_PASSWORD
- Validação da presença dos secrets para garantir que as credenciais não ficariam expostas no código.

Missão 6 — Criando o Workflow

- Criação do arquivo `.github/workflows/publish.yml` contendo os seguintes steps:
 1. Checkout do código (`actions/checkout@v4`)
 2. Definição do endereço da imagem (concatenando `REGISTRY`, `REGISTRY_NAME` e `IMAGE_NAME`)
 3. Configuração do Docker Buildx (`docker/setup-buildx-action@v3`)
 4. Geração de metadados e tags "latest" e "sha" (`docker/metadata-action@v5`)
 5. Validação de credenciais (script bash para checar se os secrets estão presentes)
 6. Login no Magalu Cloud Container Registry (`docker/login-action@v3`)
 7. Build e push da imagem (`docker/build-push-action@v6`)
 8. Resumo da execução (Summary)

Missão 7 — Primeiro Push

- Execução dos comandos para enviar o código e disparar a pipeline:
 - `git add .`
 - `git commit -m "Adiciona workflow do GitHub Actions"`
 - `git push origin main`

Missão 8 — GitHub Actions

- Acompanhamento da execução na aba Actions do GitHub.
- Troubleshooting de erros de autenticação e permissão.
- Ajuste fino no step de validação de credenciais, utilizando o bloco "env:" para garantir a leitura correta e segura das variáveis de ambiente pelo runner do GitHub.

Missão 9 — Validando a Publicação

- Acesso ao console da Magalu Cloud > Container Registry > meu-registry-01 > meu-pipeline.
- Confirmação da presença das tags "latest" e "sha-<hash>".
- Comparação do Digest (SHA256) entre o Summary do GitHub Actions e o console da Magalu Cloud.
- Digest confirmado e idêntico em ambas as plataformas:
sha256:60651c82ee6fc1e803d2f3cb0d9bcb13f8b3ca2e1a2d533c876b50e366525b97.

Missão 10 — Desafio Final

- Alteração da versão no arquivo app.py de "1.0.0" para "2.0.0".
- Novo commit e push para disparar o pipeline novamente:
 - `git add .`
 - `git commit -m "Atualiza versao da aplicacao para 2.0.0"`

- git push origin main
- Nova execução do workflow concluída com sucesso, gerando uma nova tag sha e um novo Digest.
- Validação final na VM docker-microservicos com os comandos:
 - docker login container-registry.br-se1.magalcloud
 - docker pull container-registry.br-se1.magalcloud/meu-registry-01/meu-pipeline:latest
 - docker run -d -p 5001:5000 --name app-v2 container-registry.br-se1.magalcloud/meu-registry-01/meu-pipeline:latest
 - curl http://localhost:5001
- Resultado do teste final: {"application":"Meu Pipeline","provider":"Magalu Cloud","version":"2.0.0"}

4. DESAFIOS ENCONTRADOS E SOLUÇÕES

- Erro de autenticação no git push (Windows 7): Solucionado limpando o credential.helper (git config --global credential.helper "") e utilizando um Personal Access Token com os escopos "repo" e "workflow".
- Erro de permissão ao push de arquivos em .github/workflows/: O GitHub exigiu a permissão específica de "workflow". Solucionado gerando um novo token com essa permissão habilitada.
- Secrets não lidos corretamente pelo GitHub Actions (exit code 1): O script de validação falhava ao ler a senha. Solucionado refatorando o step para usar o bloco "env:" antes do "run:", mapeando os secrets para variáveis de ambiente, o que garantiu a leitura segura e correta.
- Falha ao baixar a imagem na VM (authorization failed): O container não subia porque a VM não estava autenticada. Solucionado executando o "docker login" na VM antes do "docker pull", pois o registry é privado.

5. CONCLUSÃO

O laboratório foi concluído com 100% de sucesso. Todos os objetivos propostos foram atingidos: o Container Registry foi configurado corretamente na Magalu Cloud, as credenciais foram armazenadas de forma segura via GitHub Secrets, e a imagem Docker foi construída e publicada automaticamente a cada git push.

A rastreabilidade foi comprovada através da comparação exata dos digests (SHA256) entre o Summary do GitHub Actions e o console da Magalu Cloud. O desafio final demonstrou que o pipeline está totalmente funcional, entregando novas versões da aplicação de forma automatizada e rastreável. A prática reforçou conceitos fundamentais de DevOps, como automação, segurança de credenciais, versionamento de imagens e integração contínua.

Assinatura: sergio-l-dev-mglcloud

Data de entrega: 19 de julho de 2026